Dzanga-Sangha PPO Microphone Optimization

User manual | 10 to 160 microphones | Random, grid, hybrid and existing deployments

Purpose: use the accompanying Dzanga_PPO_10_to_160_Code.zip to evaluate microphone counts and layouts using the existing 20-elephant, 10-poacher simulation. The optimizer chooses a fixed deployment before each mission; microphones are not repositioned during the mission.

1. Package and requirements

File	Purpose
dzanga_sim.py	Existing simulation model (renamed from uploaded program).
dzanga_optimize.py	Smoke test, baseline experiment, PPO training, evaluation and plots.
requirements.txt	Python dependencies.
README.md	Concise command reference and scientific notes.

Python 3.10 or newer is recommended. Main dependencies: NumPy >= 1.24, Pillow >= 10, Gymnasium >= 1.0, Stable-Baselines3 >= 2.4, Matplotlib >= 3.8, PyTorch >= 2.2. Installation requires access to a compatible Python package index.

2. Install the program

Extract the ZIP and open a terminal in the Dzanga_PPO_10_to_160 directory. To isolate dependencies, create and activate a virtual environment:

```
python -m venv .venv
```

Windows PowerShell:

```
.venv\Scripts\Activate.ps1
```

Linux or macOS:

```
source .venv/bin/activate
```

Install the dependencies:

```
python -m pip install -r requirements.txt
```

If PowerShell blocks activation, you can instead use .venv\Scripts\python.exe for subsequent commands on Windows.

3. Smoke test

```
python dzanga_optimize.py smoke --output results/smoke
```

Runs short integration checks for all eight placement families. The smoke test verifies basic execution and coordinate export; it does not validate scientific performance or train PPO.

4. Baseline experiment: all microphone counts and layout families

```
python dzanga_optimize.py baseline --sim-seconds 86400 --dt 1 --seeds 20 --output results
```

This evaluates 31 counts (10, 15, ..., 160) times eight placement families = 248 baseline configurations. With 20 seeds, it requests 4,960 simulation episodes. The baseline uses one canonical spacing level for each family, not all five spacing variants. One simulated day per episode and a one-second timestep can be computationally expensive.

For a short initial check:

```
python dzanga_optimize.py baseline --max-cases 8 --seeds 1 --sim-seconds 120 --output results
```

Generate comparison plots after baseline CSV files are created:

```
python dzanga_optimize.py plot --output results
```

5. Train PPO

```
python dzanga_optimize.py train --timesteps 4096 --sim-seconds 86400 --dt 1 --output results
```

The policy chooses a microphone count, placement family and spacing setting through MultiDiscrete([31,8,5]). The 4,096-timestep example is an initial experiment, not evidence of convergence or optimality. Training runs headlessly, using the original model to score each deployment.

Expected saved model: results/ppo_policy.zip. This is produced only after successful training; no pretrained PPO policy is included in the package.

6. Evaluate the trained policy

```
python dzanga_optimize.py evaluate --model results/ppo_policy.zip --seeds 50 --sim-seconds 86400 --dt 1 --output results
```

Evaluate on independent random seeds. Keep the horizon and timestep identical for baseline and PPO comparisons. Examine total elephants poached, poachers caught, percentage of missions neutralizing all 10 poachers, elapsed time and microphone count. Incomplete missions must not be interpreted as successful just because no elephants were poached.

7. Search space and objective

Parameter	Choices
Microphone count	31 values: 10 to 160 inclusive, step 5
Placement families	Uniform original; Fortress original; Perimeter original; Combination original; Random; Regular grid; Jittered grid; Grid/random mix
Spacing / mixture level	0, 0.2, 0.4, 0.65, 0.9
PPO discrete choice space	31 x 8 x 5 = 1,240 configurations

The optimization reward discourages elephant poaching and uncaught poachers while favoring captures and penalizing excessive microphone deployments. PPO learns the best-performing

configuration it encounters under the simulator assumptions; a globally optimal placement is not guaranteed.

8. Output files and interpretation

Output	Meaning
baseline_runs.csv	Individual seeded baseline runs
baseline_summary.csv	Aggregated baseline results
Baseline plot (PNG)	Visual comparison produced by the plot command
ppo_policy.zip	Saved trained policy
chosen_configuration.json	Learned configuration selected for evaluation
ppo_evaluation.csv	Per-seed PPO evaluation results
ppo_deployment.csv	X,Y microphone coordinates for one evaluation seed

9. Reliability, fairness and common issues

Use --dt 1 for meaningful baseline-to-PPO comparison: the source simulation scans microphones once per simulation step when its scan interval has elapsed. Larger timesteps change effective scanning frequency. Real-world performance is not established by this model.

Use held-out seeds for PPO evaluation and sufficient replications to estimate uncertainty. Since the poacher acoustic detection radius is only 200 m, many runs may not capture every poacher within the time limit. Compare mission-completion rates, not just poaching counts.

The baseline command tests 248 canonical count/family combinations. It does not exhaustively test all 1,240 count/family/spacing combinations. Random and jittered layouts vary with the seed.

If the program cannot import gymnasium or stable_baselines3, activate the intended virtual environment and rerun installation. If PPO appears not to improve, extend training and check whether mission outcomes vary sufficiently across configurations. If runs take too long, use a short smoke test or reduce seeds and horizon for debugging only.

10. Suggested reproducible workflow

Install dependencies; run smoke; run a small baseline; confirm nontrivial outcomes; run the full baseline; train PPO; evaluate PPO on different seeds; compare counts, mission completion and poaching; examine learned coordinates on the park map. Preserve the seed list, simulator version and experiment parameters alongside reported results.

Validation note: the supplied package previously passed source compilation and smoke tests of its placement families. PPO training was not completed in the earlier environment because Gymnasium and Stable-Baselines3 were unavailable. This manual does not claim that training has converged.